
Technical University of Crete
School of Electrical and Computer Engineering

Course: Optimization

Instructor: Athanasios P. Liavas

Exercise 2 (100/1000)

Report Delivery Date: **November 18, 2025**

In the first part of the exercise, we will solve a convex quadratic optimization problems.

A. We start with a simple theoretical problem.

- (a) (10) Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$. For fixed $\mathbf{x} \in \mathbb{R}^n$ and $m > 0$, let $g_{\mathbf{x}} : \mathbb{R}^n \rightarrow \mathbb{R}$ be defined as

$$g_{\mathbf{x}}(\mathbf{y}) := f(\mathbf{x}) + \nabla f(\mathbf{x})^T(\mathbf{y} - \mathbf{x}) + \frac{m}{2} \|\mathbf{y} - \mathbf{x}\|_2^2. \quad (1)$$

- i. (5) Compute $\nabla g_{\mathbf{x}}(\mathbf{y})$.

Answer:

Let us rewrite function $g_{\mathbf{x}}(\mathbf{y})$ as

$$\begin{aligned} g_{\mathbf{x}}(\mathbf{y}) &= f(\mathbf{x}) + \nabla f(\mathbf{x})^T(\mathbf{y} - \mathbf{x}) + \frac{m}{2} \|\mathbf{y} - \mathbf{x}\|_2^2 \\ &= f(\mathbf{x}) + \nabla f(\mathbf{x})^T(\mathbf{y} - \mathbf{x}) + \frac{m}{2} (\mathbf{y} - \mathbf{x})^T \mathbf{I}(\mathbf{y} - \mathbf{x}) \end{aligned}$$

and then re-order the terms in a way that the quadratic nature of the function in terms of the variable $\mathbf{y}$ is clear

$$\begin{aligned} g_{\mathbf{x}}(\mathbf{y}) &= \frac{m}{2} (\mathbf{y} - \mathbf{x})^T (\mathbf{y} - \mathbf{x}) + \nabla f(\mathbf{x})^T (\mathbf{y} - \mathbf{x}) + f(\mathbf{x}) \\ &= \frac{m}{2} (\mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{x} - \underbrace{\mathbf{x}^T \mathbf{y}}_{\mathbf{x}^T \mathbf{y} = (\mathbf{y}^T \mathbf{x})^T = \mathbf{y}^T \mathbf{x}} + \mathbf{x}^T \mathbf{x}) + \nabla f(\mathbf{x})^T \mathbf{y} + f(\mathbf{x}) - \nabla f(\mathbf{x})^T \mathbf{x} \\ &= \frac{m}{2} (\mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T \mathbf{x} + \mathbf{x}^T \mathbf{x}) + \nabla f(\mathbf{x})^T \mathbf{y} + f(\mathbf{x}) - \nabla f(\mathbf{x})^T \mathbf{x} \\ &= \frac{m}{2} \mathbf{y}^T \mathbf{y} + \nabla f(\mathbf{x})^T \mathbf{y} - m\mathbf{x}^T \mathbf{y} + f(\mathbf{x}) - \nabla f(\mathbf{x})^T \mathbf{x} + \frac{m}{2} \mathbf{x}^T \mathbf{x} \\ &= \frac{m}{2} \mathbf{y}^T \mathbf{y} + (\nabla f(\mathbf{x}) - m\mathbf{x})^T \mathbf{y} + f(\mathbf{x}) - \nabla f(\mathbf{x})^T \mathbf{x} + \frac{m}{2} \|\mathbf{x}\|^2 \\ &= \frac{1}{2} \mathbf{y}^T (m\mathbb{I}) \mathbf{y} + (\nabla f(\mathbf{x}) - m\mathbf{x})^T \mathbf{y} + f(\mathbf{x}) - \nabla f(\mathbf{x})^T \mathbf{x} + \frac{m}{2} \|\mathbf{x}\|^2 \end{aligned}$$

This is the well-known Quadratic function with general form

$\frac{1}{2} \mathbf{x}^T \mathbf{P} \mathbf{x} + \mathbf{q}^T \mathbf{x} + r$ where $\mathbf{P} = m\mathbf{I} = m\mathbf{I}^T \succ \mathbf{O}$, $\mathbf{q} = \nabla f(\mathbf{x}) - m\mathbf{x}$ and $r = f(\mathbf{x}) + \nabla f(\mathbf{x})^T \mathbf{x} + \frac{m}{2} \|\mathbf{x}\|^2$. Obviously, $m\mathbf{I}$ is positive definite since for any vector $\mathbf{z} \neq \mathbf{0}$, it holds that $\mathbf{z}^T \cdot m\mathbf{I} \cdot \mathbf{z} = m\mathbf{z}^T \mathbf{z} = m\|\mathbf{z}\|^2 > 0$.

We already know that for a Quadratic and (strictly) convex function of the general form we presented above, its gradient is given by $\mathbf{P}\mathbf{x} + \mathbf{q}$, which in our case is:

$$\nabla g_{\mathbf{x}}(\mathbf{y}) = m\mathbf{I}\mathbf{y} + \nabla f(\mathbf{x}) - m\mathbf{x} = m\mathbf{y} + \nabla f(\mathbf{x}) - m\mathbf{x}$$

- ii. (5) Compute the optimal point, $\mathbf{y}_* = \underset{\mathbf{y}}{\operatorname{argmin}} g_{\mathbf{x}}(\mathbf{y})$, and the optimal value, $g_{\mathbf{x}}(\mathbf{y}_*)$.

Answer:

In order to find the optimal point of $g_{\mathbf{x}}(\mathbf{y})$, we just solve the equation

$$\begin{aligned} \nabla g_{\mathbf{x}}(\mathbf{y}) = 0 &\stackrel{(i)}{\Rightarrow} m\mathbf{y} + \nabla f(\mathbf{x}) - m\mathbf{x} = 0 \\ &\Rightarrow \boxed{\mathbf{y}_* = \mathbf{x} - \frac{1}{m} \nabla f(\mathbf{x})} \end{aligned}$$

Thus, the optimal value is

$$\begin{aligned} g_{\mathbf{x}}(\mathbf{y}_*) &= f(\mathbf{x}) + \nabla f(\mathbf{x})^T (\mathbf{y}_* - \mathbf{x}) + \frac{m}{2} (\mathbf{y}_* - \mathbf{x})^T \mathbf{I} (\mathbf{y}_* - \mathbf{x}) \\ &= f(\mathbf{x}) + \nabla f(\mathbf{x})^T \left(\mathbf{x} - \frac{1}{m} \nabla f(\mathbf{x}) - \mathbf{x} \right) + \frac{m}{2} \left(\mathbf{x} - \frac{1}{m} \nabla f(\mathbf{x}) - \mathbf{x} \right)^T \left(\mathbf{x} - \frac{1}{m} \nabla f(\mathbf{x}) - \mathbf{x} \right) \\ &= f(\mathbf{x}) + \nabla f(\mathbf{x})^T \left(-\frac{1}{m} \nabla f(\mathbf{x}) \right) + \frac{m}{2} \left(-\frac{1}{m} \nabla f(\mathbf{x})^T \right) \left(-\frac{1}{m} \nabla f(\mathbf{x}) \right) \\ &= f(\mathbf{x}) + \frac{-2\|\nabla f(\mathbf{x})\|^2 + m\|\nabla f(\mathbf{x})\|^2}{2m} \\ &= f(\mathbf{x}) + \frac{m-2}{2m} \|\nabla f(\mathbf{x})\|^2 \end{aligned}$$

B. We proceed to the solution of convex quadratic problems. Our goal is to study the behavior of the gradient method, with **exact** and **backtracking** line search.

(a) (40) Consider the problem

$$\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimize}} \quad f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{P} \mathbf{x} + \mathbf{q}^T \mathbf{x}, \quad (2)$$

where $\mathbf{P} \in \mathbb{R}^{n \times n}$, $\mathbf{P} = \mathbf{P}^T \succ \mathbf{O}$ and $\mathbf{q} \in \mathbb{R}^n$ (indicative values of n are $n = 2, 50, 10^2, 10^3$.)

- i. A random positive definite matrix $\mathbf{P}$ can be constructed in many ways. To fully control the condition number of the matrix, we can act as follows. Every positive definite $\mathbf{P}$ can be expressed as

$$\mathbf{P} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T \quad (3)$$

with $\mathbf{U}, \mathbf{\Lambda} \in \mathbb{R}^{n \times n}$, $\mathbf{U} \mathbf{U}^T = \mathbf{U}^T \mathbf{U} = \mathbf{I}_n$,¹ and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$, with $\lambda_i > 0$, for $i = 1, \dots, n$. The columns of $\mathbf{U}$ are the eigenvectors of $\mathbf{P}$ and the elements of the diagonal of $\mathbf{\Lambda}$ are the eigenvalues of $\mathbf{P}$.

- (a) To create a random orthonormal $\mathbf{U}$, create a random $(n \times n)$ matrix $\mathbf{A}$ as follows

$$\mathbf{A} = \text{randn}(n, n);$$

and then compute its singular value decomposition as

$$[\mathbf{U}, \mathbf{S}, \mathbf{V}] = \text{svd}(\mathbf{A});$$

Compute $\mathbf{U} * \mathbf{U}'$ and $\mathbf{U}' * \mathbf{U}$. What do you observe?

Answer:

We construct matrix $\mathbf{U}$ following the steps given in the exercise.

Upon calculating $\mathbf{U} * \mathbf{U}'$ and $\mathbf{U}' * \mathbf{U}$ we can actually see that in both cases, the two expressions return the identity matrix $\mathbf{I}_n$, with very high precision.

This verifies this exact property of the so called *Orthonormal* matrices, that the description of the exercise states that holds true.

When comparing the two expressions, **MatLab** prints in the Command Window the following:

¹Matrices that satisfy this property are called **orthogonal**.

Command Window

```

+=====+
U*U' is equal to the identity matrix!
+=====+
U'*U is equal to the identity matrix!
+=====+

```

- (b) To construct the eigenvalues λ_i , for $i = 1, \dots, n$, select the smallest and largest, $\lambda_{\min}$ and $\lambda_{\max}$, respectively. You can create the other eigenvalues in any way you wish. For example, you can create $(n - 2)$ random numbers, uniformly distributed in the interval $[\lambda_{\min}, \lambda_{\max}]$ using the command

$$\mathbf{z} = \lambda_{\min} + (\lambda_{\max} - \lambda_{\min}) * \text{rand}(n - 2, 1); \quad (4)$$

and the vector of the n eigenvalues as

$$\text{eig_P} = [\lambda_{\min}; \lambda_{\max}; \mathbf{z}]; \quad (5)$$

Matrix $\mathbf{\Lambda}$ may be constructed as

$$\mathbf{\Lambda} = \text{diag}(\text{eig_P}); \quad (6)$$

The condition number of the problem is $\mathcal{K} := \frac{\lambda_{\max}}{\lambda_{\min}}$.

- ii. (5) Construct random vector $\mathbf{q}$ and random positive definite matrix $\mathbf{P}$, with condition number $\mathcal{K}$ (indicative values, $\mathcal{K} = 10, 10^2, 10^3$).

Answer:

```

% ii.
% Construct random vector q
q = rand(n,1);

% Construct random positive definite matrix P
% using condition number K
P = U*Lambda*U.';

```

- iii. (5) Solve problem (2) using the closed-form solution, and compute the optimal point, $\mathbf{x}_*$, and the optimal value, $p_* = f(\mathbf{x}_*)$.

Answer:

From our theory, we can get the closed form solution of Quadratic functions to be

$$\mathbf{x}_* = -\mathbf{P}^{-1}\mathbf{q}$$

When calculated by **MatLab** we plug the acquired value of $\mathbf{x}_*$ in the quadratic function to find $p_* = f(\mathbf{x}_*)$.

```
% iii.  
% Find x_* using the closed form solution  
x_star = - inv(P) * q;  
  
% Calculate the optimal value, based on the closed for  
% solution  
f_star = f(x_star, P, q);
```

- iv. Solve problem (2) using the `fmincon` function of matlab.

Answer:

We use the `fmincon` function to solve the problem. We can actually verify that the optimal point of the quadratic function is given to be the same as the one given by the closed form solution, found in the previous question. Below is what is plotted in the **MatLab** Command Window:

Command Window

```
+=====+  
fmincon gives the same solution as the closed form solution!  
+=====+
```

The code realizing the preceding results is as follows

```
% iv.  
A = [];  
b = [];  
x0 = rand(n,1);  
  
% Use fmincon to minimize the function f,  
% with no constraints
```

```

x_mincon = fmincon(@(x)(1/2)*x.'*P*x + q.'*x, x0, A, b)
;

fprintf(' +=====+\n
');
if(round(x_star,4) == round(x_mincon,4))
    fprintf('fmincon gives the same solution as the
            closed form solution!\n');
end
fprintf(' +=====+\n
');

```

- v. (15) Solve problem (2) using the gradient algorithm (with exact and backtracking line search). How many iterations are sufficient for convergence, if $\mathcal{K} = 1$ and you use the exact line search?

Answer:

We implement both Line Search algorithms modified to fit the characteristics of the quadratic function that we are studying by creating the functions `Exact_LS_quadratic` and `Backtracking_LS_quadratic`.

```

function [xk,f_xk] = exact_LS_quadratic(P, q, x0,
    epsilon)

    x = x0;
    xk = [x];
    f_xk = [f(x0, P, q)];
    iters = 0;

    grad = g(x0, P, q);

    while (norm(grad) > epsilon)

        % Find t_* using the closed form
        t = (norm(grad)^2)/(grad.'*P*grad);

        % Update
        x = x - t*grad;
        xk = [xk x];
    end

```

```

        grad = g(x, P, q);
        f_xk = [f_xk f(x, P, q)];

        iters = iters + 1; % increase iteration counter
    end

end

```

```

function [xk,f_xk] = Backtracking_LS_quadratic(P, q, x0
, epsilon, alpha, beta)

x = x0;
xk = [x];

fun_val = f(x0, P, q);
f_xk = [fun_val];
grad = g(x0, P, q);

iters = 0;

while (norm(grad) > epsilon)
    iters = iters + 1;
    t = 1;

    while (fun_val - f(x - t*grad, P, q) < alpha*
        t*norm(grad)^2)
        t = beta * t;
    end

    % Update
    x = x - t*grad;
    xk = [xk x];

    fun_val = f(x, P, q);
    f_xk = [f_xk fun_val];
    grad = g(x, P, q);

end

```

end

For $\mathcal{K} \neq 1$ we can notice that both Exact and Backtracking Line Search need a certain number of iterations, which is always greater than 1 (the dependency of this number to $\mathcal{K}$ will be studied later on).

However, in the case of $\mathcal{K}$ being 1, the number of iterations needed for convergence by both algorithms is always 1.

The above phenomenon can be explained upon examining the plots for the case of $\mathcal{K} = 1$

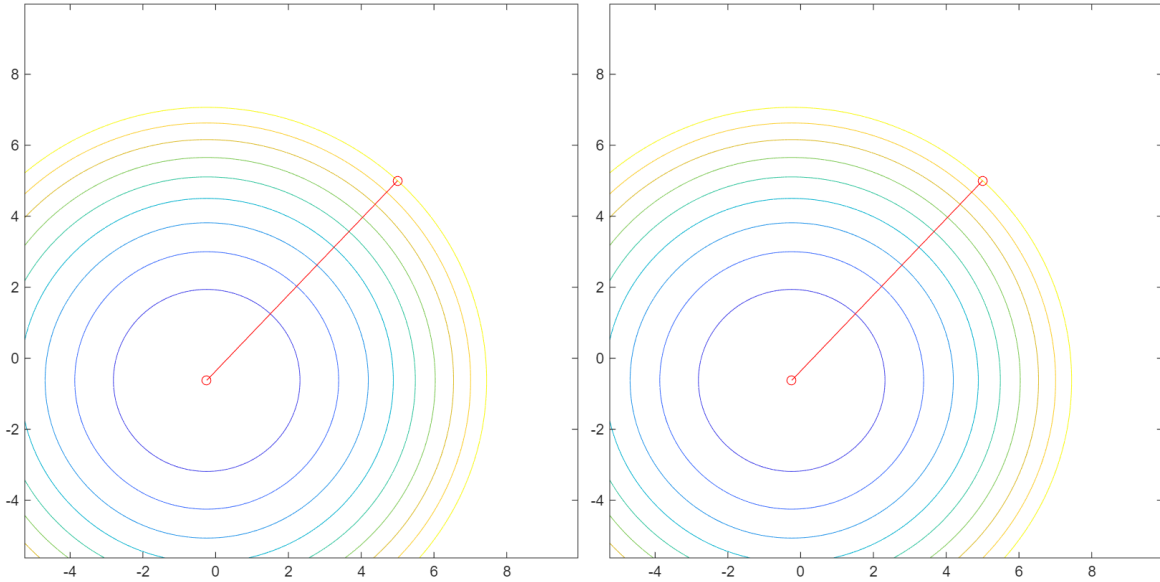


Figure 1: Contour plots of the convergence paths followed by GD with ELS (left) and BT-LS (right), for $\mathcal{K} = 1$

We can see that since the level sets of f are spheres in the 2D space, the gradient of f at any point on them (being perpendicular to their tangents) will point directly to the minimum of the function.

This means that in both ELS and BT-LS the next point of the iterative procedure ($\mathbf{x}_{k+1}$) is just the minimum point, thus making the needed iterations for convergence just 1.

- vi. (5) For $n = 2$, construct a contour plot, with levels the values $f(\mathbf{x}_k)$. Then, on top of this plot, plot the trajectories of $\{\mathbf{x}_k\}$ produced by the two algorithms. (For problems with large condition number, you must observe the “zig-zag” effect.)

Answer :

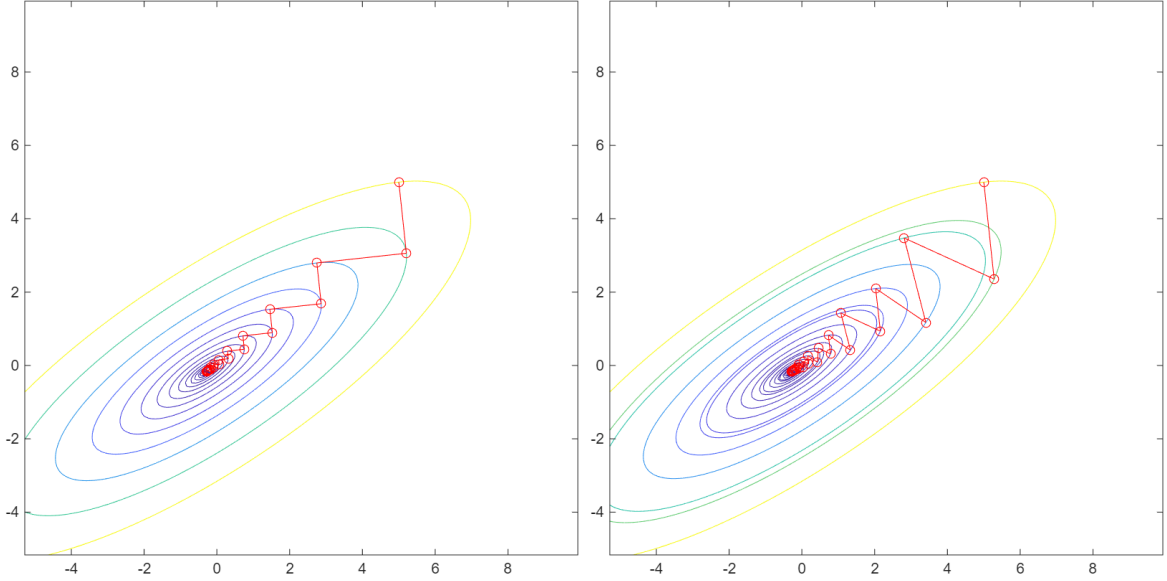


Figure 2: Contour plots of the convergence paths followed by GD with ELS (left) and BT-LS (right), for $\mathcal{K} = 10$

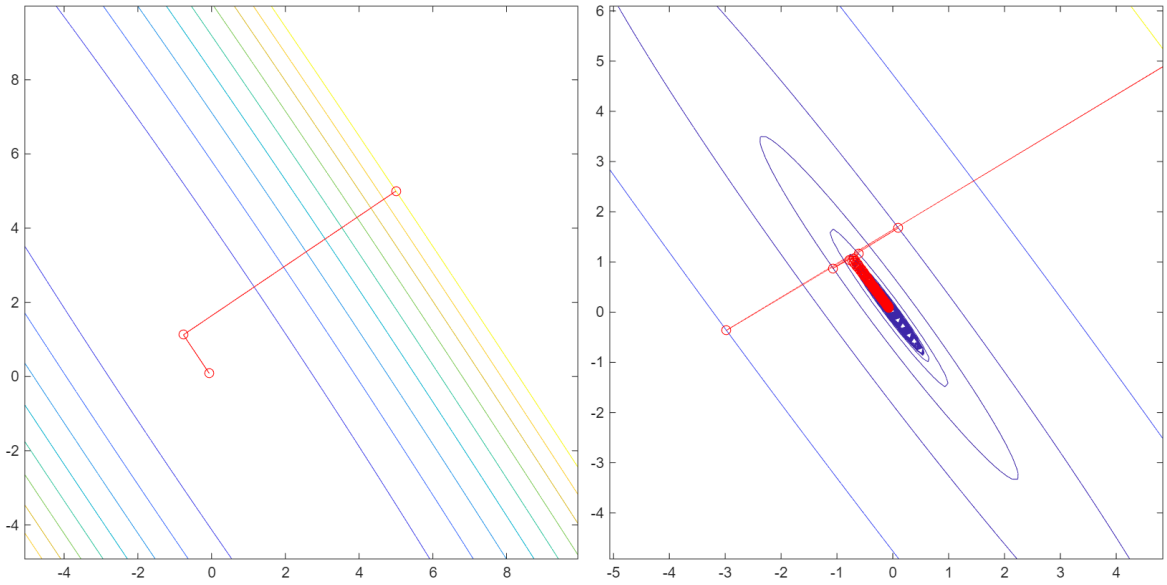


Figure 3: Contour plots of the convergence paths followed by GD with ELS (left) and BT-LS (right), for $\mathcal{K} = 100$

The code that produced the preceding results is as follows

```
if (n == 2)
    step = 0.1;
    x1 = min(x0(1), x_star(1)) - 5:step:max(x0(1),
        x_star(1)) + 5;
    x2 = min(x0(2), x_star(2)) - 5:step:max(x0(2),
        x_star(2)) + 5;
    for i1 = 1:length(x1)
        for i2 = 1:length(x2)
            x_tmp = [x1(i1); x2(i2)];
            f_(i1,i2) = f(x_tmp, P, q);
        end
    end

figure
if (length(F_exact) < 20)
    ls = linspace(F_exact(1),F_exact(end),10);
    contour(x1, x2, f_', ls);
    axis('square'); % plot the level sets of
        f_duadratic at the positions x_k
    hold on
    plot(X_exact(1,:), X_exact(2,:), '-or'); % plot
        the trajectory of x_k
    hold off
else
    contour(x1, x2, f_', sort(F_exact));
    axis('square'); % plot the level sets of
        f_duadratic at the positions x_k
    hold on
    plot(X_exact(1,:), X_exact(2,:), '-or'); % plot
        the trajectory of x_k
    hold off
end

figure
if (length(F_bt) == 2)
    ls = linspace(F_exact(1),F_exact(end),10);
    contour(x1, x2, f_', ls);
```

```

axis('square'); % plot the level sets of
    f_duadratic at the positions x_k
hold on
plot(X_bt(1,:), X_bt(2,:), '-or'); % plot the
    trajectory of x_k
hold off
else
    contour(x1, x2, f_', sort(F_bt));
    axis('square'); % plot the level sets of
        f_duadratic at the positions x_k
    hold on
    plot(X_bt(1,:), X_bt(2,:), '-or'); % plot the
        trajectory of x_k
    hold off
end
end

```

As pointed out in earlier answers, the two algorithms need a certain number of iterations in order to get ϵ -close to the optimal point, after following an iterative procedure.

The paths followed by each algorithm, in each iteration, is denoted using the red markers.

As we can easily see the number of iterations (number of "zig-zags") increases, as $\mathcal{K}$ gets larger, with the effect being more and more visible as it attains larger and larger values.

This can be interpreted in the following way.

$\mathcal{K}$ is defined as the ratio of the largest and smallest eigenvalue of matrix $\mathbf{P}$. The larger this ratio is, the more the two eigenvalues of matrix $\mathbf{P}$ differ, making the form of the quadratic function more "skewed" and steep in certain directions, since $\mathbf{P}$ "controls" the form of the quadratic term (as a does in the 1D case of $ax^2 + bx + c$; bigger a means a "wider" quadratic). Considering all the above, the plotted results are to be expected, since a more uneven quadratic will unavoidably result in less smooth ("Gradient-based") path followed by the two algorithms.

- vii. (5) Plot quantity $\log(f(\mathbf{x}_k) - p_*)$, produced by the two algorithms, versus k . What is the slope of this plot? What do you observe for different condition numbers?

Answer:

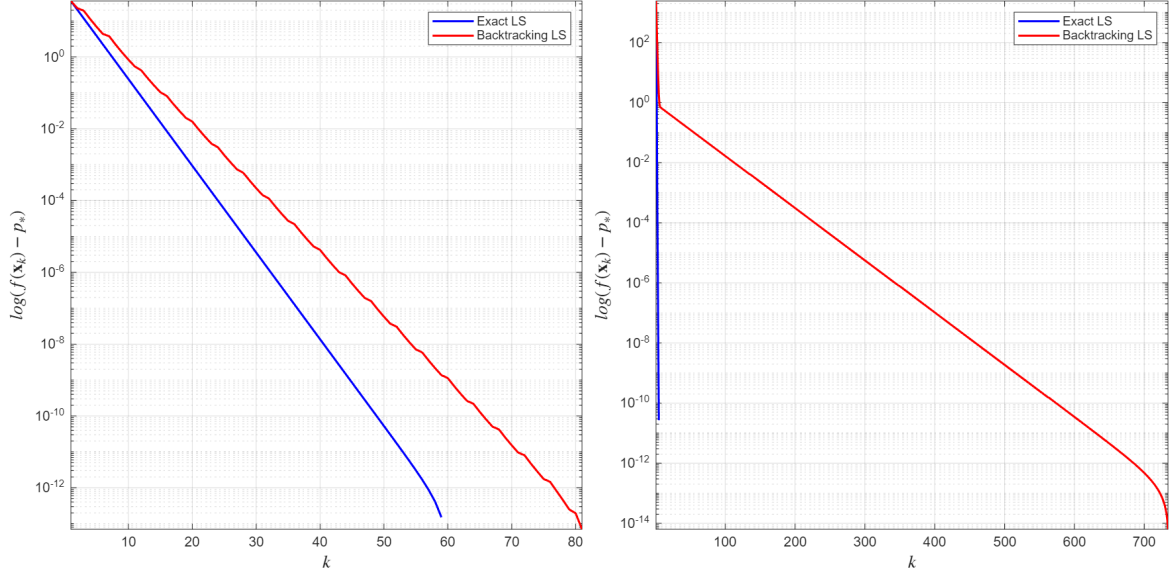


Figure 4: Common plot of $\log(f(\mathbf{x}_k) - p_*)$ vs k for ELS and BT-LS, for $n = 2$ and $\mathcal{K} = 10$ (left) and $\mathcal{K} = 100$ (right)

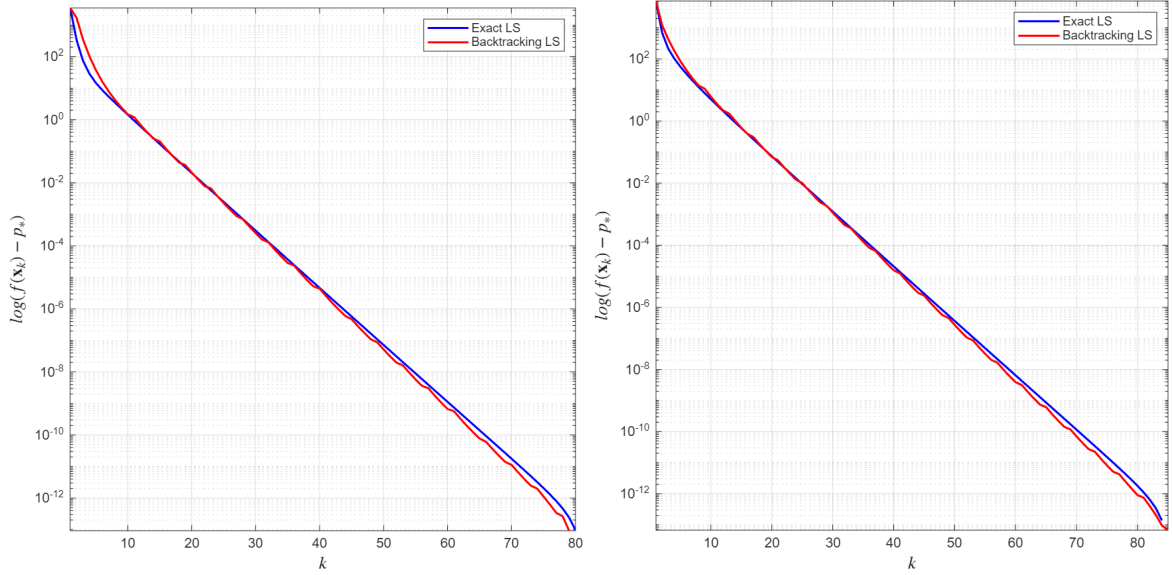


Figure 5: Common plot of $\log(f(\mathbf{x}_k) - p_*)$ vs k for ELS and BT-LS, for $\mathcal{K} = 10$ and $n = 50$ (left) and $n = 100$ (right)

The code that produced the results above

```
% vii.
figure
semilogy([1:length(F_exact)]', (F_exact-F_exact(end)),
         'b', 'LineWidth', 1.5);
hold on;
semilogy([1:length(F_bt)]', (F_bt-F_bt(end)), 'r', '
         LineWidth', 1.5);
grid on;
xlabel('$k$', 'FontSize', 13, 'Interpreter','latex');
ylabel('$\log(f(\mathbf{x}_k)- p_*)$', 'FontSize', 13, '
         Interpreter', 'latex');
legend('Exact LS', 'Backtracking LS');
axis tight
```

We can visually verify, from the upper set of **MatLab** figures, that for a fixed small number of dimensions n , LS always terminates in much less iterations compared to BT-LS, for any value of $\mathcal{K}$.

However, this does not hold when increasing the number of dimensions n and having a fixed condition number $\mathcal{K}$; this becomes clear upon checking the lower set of figures attached. It can be said that as n increases, ELS “loses” its advantage over BT-LS of terminating in less iterations, as their graphs tend to fit exactly one on to another.

When it comes to calculating the slope of the plot, which is defined as the quantity $\log c$, where $c = 1 - \frac{1}{\mathcal{K}}$, we can see the results that **MatLab** yields in its Command Window, for $\mathcal{K} = 1, 10$ and 100 respectively

Command Window
<pre>+=====+ The slope of the plot for $\mathcal{K} = 1$ is: $-\infty$ +=====+</pre>

Command Window

```
+=====+  
The slope of the plot for  $\mathcal{K} = 10$  is: -0.046  
+=====+
```

Command Window

```
+=====+  
The slope of the plot for  $\mathcal{K} = 100$  is: -0.004  
+=====+
```

The code that yielded the results above is

```
% Calculate the slope of the plot  
c = 1 - (1/K);  
slope = log10(c);  
fprintf('The slope of the plot for K=%d is: %.3f\n', K,  
        slope);  
fprintf(' +=====+\n  
        ');
```

As we can see, for larger condition numbers, the number of iterations
This behavior of the slope values is to be expected since the slope of the
plot for $\mathcal{K} \rightarrow \infty$ then $c = 1 - \underbrace{\frac{1}{\mathcal{K}}}_{\rightarrow 0}$ which means that the slope $\log c$ tends
to 0.

On the other hand, if $\mathcal{K} \rightarrow 1$ then $c \rightarrow 0$ which leads to the slope
 $\log c \rightarrow -\infty$

- viii. (5) Using the convergence analysis results for strongly convex functions, and the values of $f(\mathbf{x}_0)$, p_* and ϵ , compute the minimum number of iterations that guarantees solution within accuracy ϵ , i.e., $f(\mathbf{x}_k) - p_* \leq \epsilon$, and compare it with the number of iterations performed by the algorithms. What do you observe?

Answer:

Command Window

```
+=====+
Max #iters needed for  $\epsilon$ -close convergence, with  $\mathcal{K} = 1$ , is: 17.30
+=====+
```

Command Window

```
+=====+
Max #iters needed for  $\epsilon$ -close convergence, with  $\mathcal{K} = 10$ , is: 193.61
+=====+
```

Command Window

```
+=====+
Max #iters needed for  $\epsilon$ -close convergence, with  $\mathcal{K} = 100$ , is: 1860.75
+=====+
```

The code that produced the results above

```
% viii.
k_epsilon = K*log((f(x0,P,q) - f_star) / epsilon);
fprintf('Max #iters needed for 10^{(d)}-close
        convergence, with K=%d, is: %.2f\n', log10(epsilon),
        K, k_epsilon);
fprintf(' +=====+ \n
        ');
fprintf(' \n');
```

As we can see from the results, the maximum number of iterations that may be ran, in order for the algorithms to converge, is roughly getting 10 times bigger from its previous value, when $\mathcal{K}$ gets 10 times bigger too.

This reveals that k_ϵ is proportional to $\mathcal{K}$.

In fact we know this approximate equality that holds, from our theory

$$k_\epsilon \approx \mathcal{K} \log \left(\frac{f(\mathbf{x}_0) - p_*}{\epsilon} \right)$$

which fully encapsulates our observation and backs it up from theoretical perspective.

In the second part of this exercise, we will study the problem of (linear) data classification via Logistic Regression (LR). For the notation, please, check my Lecture Notes (50).

A. Data Generation: We generate the (approximately) linearly separable dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i \in \mathbb{R}^N$ and $y_i \in \{1, 0\}$. The data generation I propose proceeds as follows:

1. Generate random $\mathbf{w} \in \mathbb{R}^N$, and $b \in \mathbb{R}$.

Answer:

Implemented by the following code:

```
% 1.
w = rand(N, 1);
b = rand;
```

2. (5) Compute a point, say $\mathbf{x}_0$, of the hyperplane $\mathbb{H}_{b,\mathbf{w}}$.

Answer:

Since we want a point of $\mathbb{R}^n$ such that $\mathbf{x}_0 \in \mathbb{H}_{b,\mathbf{w}}$, we derive the equation

$$\mathbf{w}^T \mathbf{x}_0 = b$$

which easily gives us that

$$||\mathbf{w}||^2 \mathbf{x}_0 = b\mathbf{w}$$

or equivalently

$$\mathbf{x}_0 = \frac{b}{||\mathbf{w}||^2} \mathbf{w}$$

Thus, by generating random $\mathbf{w}$ and b as done earlier, we can easily compute a point $\mathbf{x}_0$ of the Hyperplane $\mathbb{H}_{b,\mathbf{w}}$.

```
% 2.
% x_0 exists in the Hyperplane: H = {w^T * x = b}
x0 = (w/norm(w).^2) .* b;
```

3. For $i = 1, \dots, n$, generate random class $y_i \equiv y(i) \in \{0, 1\}$.

Answer:

Implemented by the following code:

```
% 3.  
% Randomly generate the {0,1} class labels  
y = zeros(1,n);  
for i=1:n  
    y(i) = round(rand);  
end
```

4. For $i = 1, \dots, n$, generate the data points $\mathbf{x}_i \equiv \mathbf{X}(:, i)$ as follows:

$$\mathbf{x}_i = \begin{cases} \mathbf{x}_0 + \mathbf{w} + \text{std} * \text{randn}(N, 1), & \text{if } y_i = 1, \\ \mathbf{x}_0 - \mathbf{w} + \text{std} * \text{randn}(N, 1), & \text{otherwise,} \end{cases} \quad (7)$$

where parameter **std** is the standard deviation of the noise and controls the linear separability of the data. “Small” **std** leads, with high probability, to separable data, while the opposite happens for “large” **std**.

Answer:

Implemented by the following code:

```
% 4.  
% Randomly generate the data points in the N-  
    dimensional space  
X = zeros(N,n);  
std = 0.75;  
  
for i=1:n  
    if (y(i) == 1)  
        X(:,i) = x0 + w + std * randn(N,1);  
    else  
        X(:,i) = x0 - w + std * randn(N,1);  
    end  
end
```

5. (10) If $N = 2$, plot the line defined by $\mathbf{w}$ and b , as well as the data points $\mathbf{x}_i$, for $i = 1, \dots, n$, each class with different color. Check, visually, the separability of the data and perform experiments with different values of $\mathbf{w}$, b and std .

Answer:

For fixed values of $\mathbf{w}$ and b , we loop through the previous steps of generating the data, for varying values of standard deviation σ^2 , making the data more and more spaced out in the 2D plane. The code implementing the plotting is as follows:

```
% 5.
step = 0.01;
x1 = x0(1) - 5: step: x0(1) + 5-step;
x2 = x0(2) - 5: step: x0(2) + 5-step;
x = [x1; x2];

[a,c] = line_func_2D(w, b, x0);
sep_line = a*x + c;

if (N == 2)
    % Plot the data points and the separating
    % hyperplane
    figure;
    scatter(X(1, y == 0), X(2, y == 0), 'r', 'filled'); % Class 0
    hold on;
    scatter(X(1, y == 1), X(2, y == 1), 'b', 'filled'); % Class 1
    hold on;

    plot(x(1:10:end), sep_line(1:10:end), 'Color', 'k', 'LineStyle', ':');
    hold on;
    plot(x0(1), x0(2), 'Marker', 'o', 'MarkerEdgeColor', 'm', 'MarkerFaceColor', 'm', 'MarkerSize', 5);
    text(x0(1) + 0.01, x0(2) + 0.01, 'x_0', 'FontSize', 12, 'Color', 'k', 'FontWeight', 'b');
```

```

        bold');

xlabel('x_1','FontSize',13);
ylabel('x_2','FontSize',13);
x1_min = min(X(1,:))-0.2; x1_max = max(X(1,:))
        +0.2; x2_min = min(X(2,:))-0.2; x2_max = max
        (X(2,:))+0.2;
axis([x1_min x1_max x2_min x2_max]);
lgd = legend('Class 0', 'Class 1');
title(lgd, 'True Labels');
grid on;
hold off;
title(['Seperable Dataset with  $\sigma^2=$ ',
        num2str(std)] , 'Interpreter','latex');
end

```

6. Define $\theta := (b, \mathbf{w})$ and augment appropriately the data points by inserting -1 at their first position. You may call the new data matrix as $\mathbf{X}_{\text{augm}}$.

Answer:

Implemented by the following code:

```

% 6.
% Define vector theta := (b,w)
theta_0 = [b; w];

% Augment dataset X
X_augm = zeros(N+1,n);

for i=1:n
    X_augm(:,i) = [-1; X(:,i)];
end

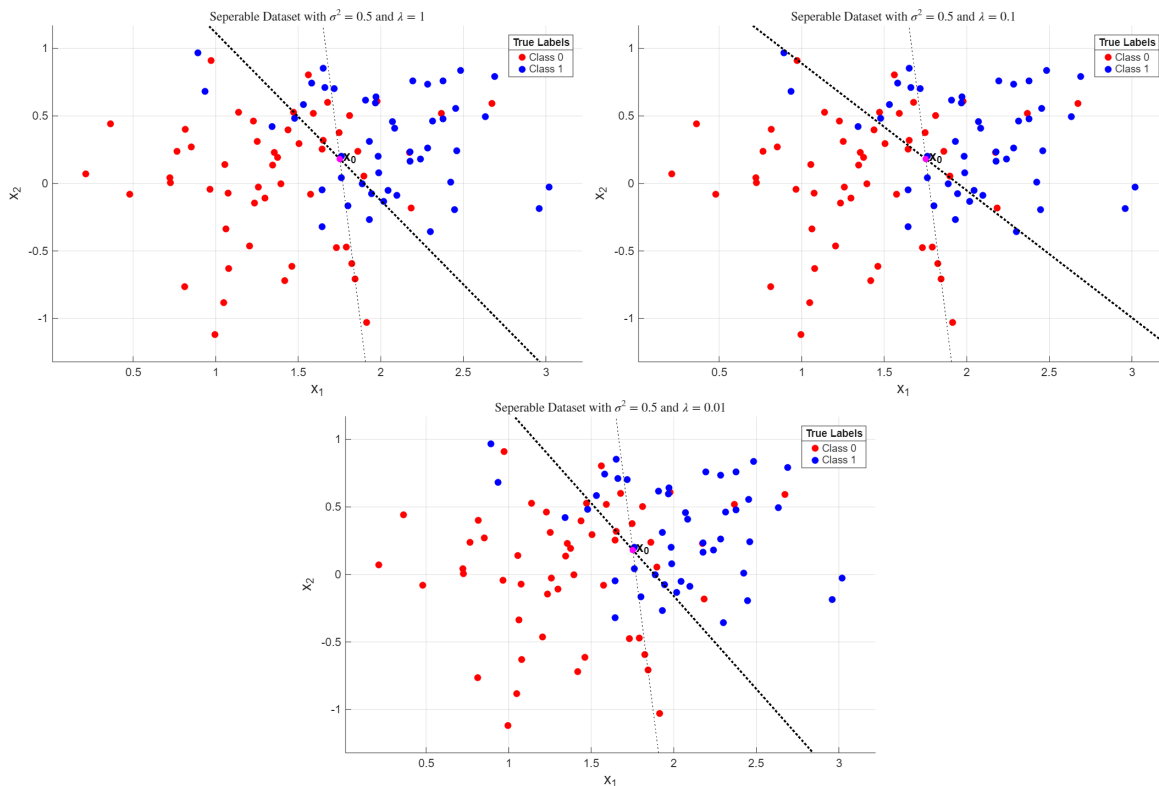
```

B. Classification via Logistic Regression:

1. (5) Choose $\lambda > 0$ (for example, $\lambda = 10^0, 10^{-1}, 10^{-2}$) and solve the l_2 -regularized LR problem via the `fmincon`. Denote the solution as `theta_*`. If $N = 2$, plot the data, the original line used for data generation, and the line computed via the LR. Perform experiments with different values of λ . What do you observe?

Answer:

By running our simulations for different values of parameter λ and fixed b , $\mathbf{w}$ and σ^2 we get the following plots



The code used to get the preceding results is the following

```
% 1.
% Iterate through (positive) values of parameter lambda
lambda = 10^0;
while (lambda > 10^(-2))

    % Define null constraints for fmincon
    A = [];
```

```

b = [];

% Define the initial theta of the LR
theta_star = fmincon(@(theta)
(-1/n)*sum( y*log(1./(1+exp(-X_augm.'*theta))) ...
+ (1-y)*log(1-(1./(1+exp(-X_augm.'*theta)))) ) ...
+ lambda .* (1/2)*norm(theta)^2, theta_0, A, b);

if (N == 2)
    b_star = theta_star(1);
    w_star = theta_star(2:3);

    [a,c] = line_func_2D(w_star, b_star, x0);
    sep_line_opt = a*x + c;

    % Plot the data points and the separating
    % hyperplane
    figure;
    scatter(X(1, y == 0), X(2, y == 0), 'r', '
        filled'); % Class 0
    hold on;
    scatter(X(1, y == 1), X(2, y == 1), 'b', '
        filled'); % Class 1
    hold on;

    plot(x(1:10:end), sep_line(1:10:end), 'Color',
        'k', 'LineStyle', ':');
    hold on;
    plot(x(1:10:end), sep_line_opt(1:10:end), '
        Color', 'k', 'LineStyle', ':', 'LineWidth',
        1.7);
    hold on;
    plot(x0(1), x0(2), 'Marker', 'o', '
        MarkerEdgeColor', 'm', 'MarkerFaceColor', 'm
        ', 'MarkerSize', 5);
    text(x0(1) + 0.01, x0(2) + 0.01, 'x_0', '
        FontSize', 12, 'Color', 'k', 'FontWeight', '
        bold');

```

```

    lgd1 = legend('Initial seperating line', '
        Optimal seperating line');
    hold on

    xlabel('x_1','FontSize',13);
    ylabel('x_2','FontSize',13);
    x1_min = min(X(1,:))-0.2; x1_max = max(X(1,:))
        +0.2;

    x2_min = min(X(2,:))-0.2; x2_max = max(X(2,:))
        +0.2;
    axis([x1_min x1_max x2_min x2_max]);
    lgd2 = legend('Class 0', 'Class 1');
    title(lgd2, 'True Labels');
    grid on;
    hold off;
    title(['Seperable Dataset with  $\sigma^2=$ ',
        num2str(std), ' and  $\lambda=$ ', num2str(
            lambda)] , 'Interpreter','latex');

end

lambda = 0.1*lambda;
end

```

As we can see, by using smaller and smaller values for λ , the Logistic Regression algorithm gives a different separating Hyperplane tries as being the optimal.

There is no real tendency (for example the slope of the separating line increasing as λ decreases); the LR algorithm just tries to find an optimal slope in a way that it minimizes the incorrect guesses regarding the classes of the existing data points.

This is something to be expected since, from our theory, we know that the value of the λ coefficient is acquired through validation techniques and not an actual formula.

2. (20) Solve the l_2 -regularized LR problem via Gradient Descent (GD) with Backtracking line search. Use any terminating condition you think is appropriate. In the k -th iteration, compute the “error” `norm(theta_k - theta_*)`.

Answer:

When trying to solve an instance of the LR problem using the GD algorithm with Backtracking LS we get the following in the **MatLab** Command Window:

```

Command Window

+=====+
BackTracking LS applied for the optimal of J_R(theta)

-----

Relative Percentage Error: 0.511756 [%]
Relative Percentage Error: 0.179205 [%]
Relative Percentage Error: 0.065178 [%]
Relative Percentage Error: 0.025133 [%]
Relative Percentage Error: 0.009690 [%]
Relative Percentage Error: 0.003763 [%]
Relative Percentage Error: 0.001460 [%]
Relative Percentage Error: 0.000567 [%]
Relative Percentage Error: 0.000220 [%]
Relative Percentage Error: 0.000086 [%]
Relative Percentage Error: 0.000033 [%]
Relative Percentage Error: 0.000013 [%]
Relative Percentage Error: 0.000005 [%]
Relative Percentage Error: 0.000002 [%]
Relative Percentage Error: 0.000001 [%]

The GD algorithm with BT-LS found
the solution of the LR problem to be the same as fmincon()!
theta_* = [0.071 0.283 0.210]
+=====+

```

We implemented the GD algorithm with BT-LS and present the respective code below

```
% 2.
fprintf('++++++++++++++++++++++++++++++++++++\n
      ');
fprintf('BackTracking LS applied for the optimal of J_R
      (theta)\n');
fprintf('-----\n'
      );
theta_opt = Backtracking_LS_LR(X_augm, y, theta_0,
      lambda, epsilon, alpha, beta, n, theta_star);
fprintf('++++++++++++++++++++++++++++++++++++\n
      ');
```

```
function [theta_opt] = Backtracking_LS_LR(X_augm, y,
      theta_0, lambda, epsilon, alpha, beta, n, theta_star
      )

theta = theta_0;
grad = grad_J_R(X_augm, theta, lambda, y, n);
fun_val = J_R(X_augm, theta, lambda, y, n);

iters = 0;

while (norm(grad) > epsilon)
    iters = iters + 1;
    t = 1;

    while (fun_val - J_R(X_augm, theta - t*grad, lambda
        , y, n) < alpha*t*norm(grad)^2)
        t = beta*t;
    end

    theta = theta - t*grad;
    fun_val = J_R(X_augm, theta, lambda, y, n);
    grad = grad_J_R(X_augm, theta, lambda, y, n);

    fprintf('Relative Percentage Error: %.6f [%%]', (
```

```

        norm(theta - theta_star)/norm(theta_star) ) );
    fprintf('\n');
end

theta_opt = theta;
if (theta_star - epsilon <= theta_opt & theta_opt <=
    theta_star + epsilon)
    fprintf('\n\n');
    fprintf('The GD algorithm with BT-LS found the
        solution of the LR problem to be the same as
        fmincon()!\n');
    fprintf('theta_* = [%.3f %.3f %.3f]\n', theta_opt
        (1), theta_opt(2), theta_opt(3));
end

end

```

As we can see BT-LS converges pretty fast, even if we run it for different parameters of the LR problem.

The (strongly) convex behavior, thus our ability to actually use the GD BT-LS algorithm, has a lot to do with the addition of the regularizer function $R(\theta)$ to the cost function $J(\theta)$, in previous steps, allowing us to work with the strongly convex function $J_R(\theta)$.